شرح مشروع نظام إدارة الحضور

Attendance Management System · COMP 301 · شرح كل كلاس وكل حاجة بتعملها بالتفصيل

الملف ده لمين؟ ليكم انتو الأربعة عشان تفهموا المشروع كويس قبل التسليم والمناقشة. فيه: فكرة المشروع، تقسيم الكود، شرح كل كلاس سطر بسطر تقريبًا، شرح الشاشات، سيناريو كامل بيحصل جوه البرنامج، شرح خطوط الـ UML، وفي الآخر أسئلة متوقعة في المناقشة وإجابتها.

1. فكرة المشروع في كلمتين

البرنامج بيساعد الكلية تسجّل حضور الطلبة إلكترونيًا بدل الورق. فيه 3 أنواع من الناس:

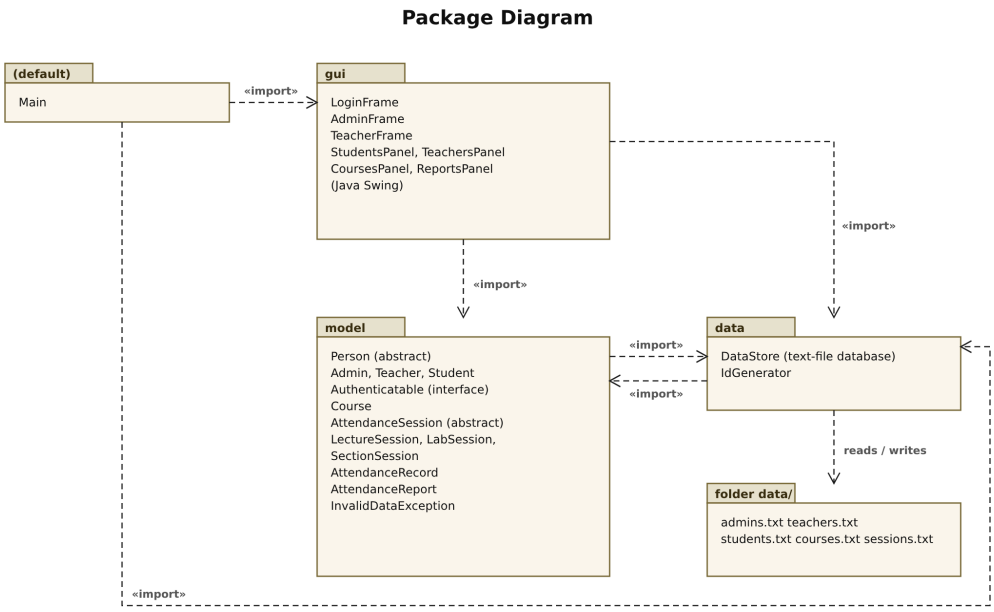
- **الأدمن (Admin):** هو اللي بيضيف الطلبة والمدرسين والكورسات، ويحدد مين المدرس بتاع كل كورس، ويسجّل الطلبة في الكورسات.
- **المدرس (Teacher):** بيدخل بالإيميل والباسورد، يختار كورس من كورساته، يختار النوع (محاضرة / لاب / سكشن)، وبعدين كل طالب يقوله رقمه فيكتبه، وفي الآخر يدوس End Session.
- **الطالب (Student):** مش بيدخل على البرنامج خالص. هو بس عنده رقم حضور من 4 أرقام بيقوله للمدرس.

وكل واحد عنده ID عشوائي ومش متكرر: الطالب 4 أرقام، المدرس 5 أرقام، الأدمن 6 أرقام. والبيانات كلها بتتحفظ في ملفات نصية جوه فولدر `data` فلما تقفل البرنامج وتفتحه تلاقيها زي ما هي.

2. شكل المشروع من جوه

```
Program/                      <- ده فولدر المشروع اللي بتفتحوه في IntelliJ
├── src/
│   ├── Main.java             <- بداية البرنامج
│   ├── model/                <- كلاسات المشكلة نفسها (الناس والكورسات والحضور)
│   │   ├── Person.java  Admin.java  Teacher.java  Student.java
│   │   ├── Authenticatable.java   (interface)
│   │   ├── Course.java
│   │   ├── AttendanceSession.java  LectureSession.java  LabSession.java  SectionSession.java
│   │   ├── AttendanceRecord.java  AttendanceReport.java
│   │   └── InvalidDataException.java
│   ├── data/                 <- الداتا بيز": الحفظ والتحميل وتوليد الأرقام"
│   │   ├── DataStore.java
│   │   └── IdGenerator.java
│   └── gui/                  <- الشاشات (Swing)
│       ├── LoginFrame.java  AdminFrame.java  TeacherFrame.java
│       └── StudentsPanel.java  TeachersPanel.java  CoursesPanel.java  ReportsPanel.java
├── data/                     <- بيتعمل لوحده أول ما تشغّل (فيه ملفات الـ txt)
└── docs/                     <- الـ PDFs وصور الـ UML
```

التقسيم ده اسمه **layers**: الشاشات (gui) مش بتعمل أي حسابات، هي بس بتاخد الكلام من المستخدم وتنادي الـ model. والـ model مش عارف حاجة عن الشاشات. وده بيخلي الكود مترتب وسهل تشرحوه.

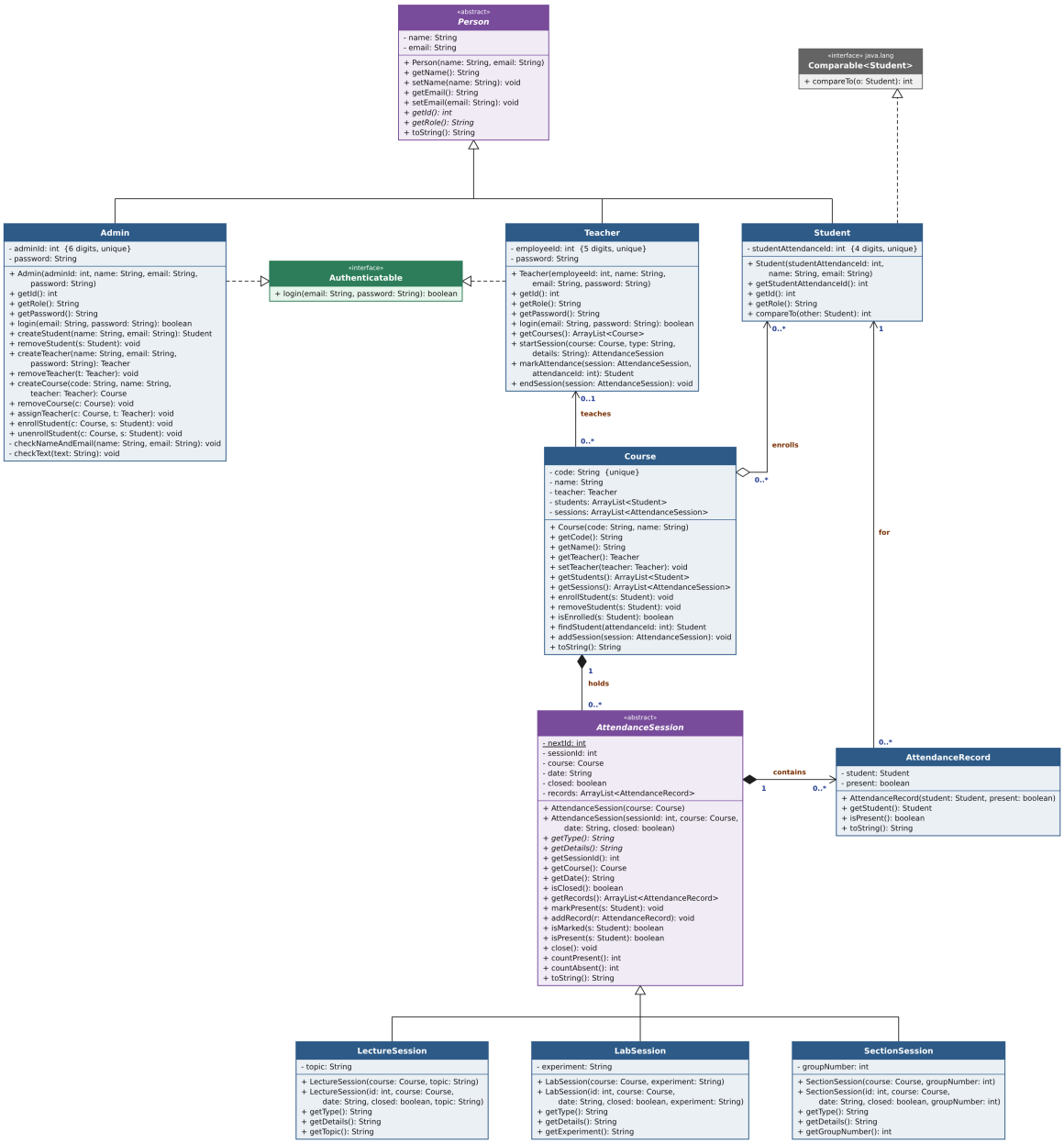


Package diagram: مين بيستخدم مين

3. الـ package اللي اسمها model

ده قلب المشروع. هنمشي على كل كلاس: هو إيه، ليه موجود، فيه إيه، وكل method بتعمل إيه.

Class Diagram 1 - Domain Model (package model)



Class Diagram 1 (موجود بحجم كبير في ملف الـ UML الإنجليزي وفي فولدر docs/uml)

Person 3.1 `abstract class`

ليه موجود؟ الأدمن والمدرس والطالب كلهم عندهم اسم وإيميل. فبدل ما نكرر الحاجتين دول في 3 كلاسات، حطيناهم في أب واحد اسمه `Person` والتلاتة بيعملوا منه `extends` (ده الـ Inheritance من محاضرة 4).

ليه abstract؟ لأن مفيش حاجة في الحقيقة اسمها "شخص وخلاص"؛ أي شخص في السيستم لازم يكون أدمن أو مدرس أو طالب. فمش منطقي نعمل `new Person(...)`، والـ abstract بيمنع ده (محاضرة 5).

العضو	بيعمل إيه
`- name: String` , `- email: String`	private عشان الـ Encapsulation؛ محدش يغيّرهم غير من خلال الـ setters.
`+ Person(name, email)`	الكونستركتور. الأبناء بينادوه بـ `super(name, email)` .
`getName / setName / getEmail / setEmail`	getters و setters عادية.
`+ getId(): int` **abstract**	كل ابن بيرجّع الـ ID بتاعه. ليه abstract؟ لأن كل واحد الـ ID بتاعه اسمه مختلف وعدد أرقامه مختلف (adminId / employeeId / studentAttendanceId).
`+ getRole(): String` **abstract**	بترجع "Admin" أو "Teacher" أو "Student".
`+ toString()`	override لـ toString بتاعة كلاس Object (محاضرة 2). بترجع مثلًا `3117 - Mohamed Ali`، وده اللي بيظهر في الـ JList والـ JComboBox أوتوماتيك.

```
public abstract int getId();
```

```
public String toString() {
    return getId() + " - " + name;
}
```

Authenticatable 3.2 `interface`

interface فيه method واحدة: `boolean login(String email, String password)` . الأدمن والمدرس بس اللي بيعملوه `implements` . الطالب لا. وده معناه في التصميم إن **الطالب مينفعش يعمل login**، وده بالظبط اللي انتو عايزينه.
(Interfaces من محاضرة 5)

```
package model;

/**
 * Interface for the users that are allowed to log in (Admin and Teacher only).
 * A Student does not implement it, so a student can never log in.
 * (Lecture 5 - Interfaces)
 */
public interface Authenticatable {

    /** Returns true if the given email and password belong to this user. */
    boolean login(String email, String password);
}
```

Admin 3.3 `extends Person implements Authenticatable`

الأدمن عنده `adminId` من 6 أرقام و `password` . وهو "المدير" اللي بيعمل كل حاجة تخص البيانات:

Method	بتعمل إيه بالظبط
`Admin(adminId, name, email, password)`	كونستركتور واحد بس. الـ ID بيتبعتله جاهز (إما جديد من IdGenerator أو متقري من الملف).
`getId()` , `getRole()`	override للـ abstract methods: ترجع adminId و "Admin".
`login(email, password)`	ترجع true لو الإيميل (من غير ما تفرق capital/small) والباسورد صح.
`createStudent(name, email)`	تتأكد من الاسم والإيميل، تاخد رقم 4 خانات جديد من `IdGenerator.newStudentId()` ، تعمل Student، وتضيفه في DataStore (اللي بيحفظ على طول). لو فيه غلط ترمي `InvalidDataException` .
`createTeacher(name, email, password)`	نفس الفكرة بس بـ ID من 5 أرقام، والباسورد لازم 4 حروف على الأقل.
`createCourse(code, name, teacher)`	تتأكد إن الكود والاسم مش فاضيين وإن الكود مش متكرر، تعمل Course وتحط المدرس بتاعه.
`assignTeacher(c, t)`	تغيّر مدرس الكورس وتحفظ.
`enrollStudent(c, s)` / `unenrollStudent(c, s)`	تسجّل الطالب في الكورس أو تشيله، وتحفظ.
`removeStudent / removeTeacher / removeCourse`	تمسح من DataStore (وهو بيشيل الطالب من كل الكورسات، أو يفضّي مكان المدرس في كورساته).
`checkNameAndEmail` (private)	اسم مش فاضي، إيميل فيه @ و نقطة، ومش مستخدم قبل كده.
`checkText` (private)	تمنع علامتي `;` و `,` لأننا بنستخدمهم كفواصل جوه ملفات الداتا.

```
public Student createStudent(String name, String email) throws InvalidDataException {
    checkNameAndEmail(name, email);
    Student s = new Student(IdGenerator.newStudentId(), name.trim(), email.trim());
    DataStore.addStudent(s);
    return s;
}
```

```
private void checkNameAndEmail(String name, String email) throws InvalidDataException {
    if (name == null || name.trim().isEmpty()) {
        throw new InvalidDataException("Name is required.");
    }
    if (email == null || !email.contains("@") || !email.contains(".")) {
        throw new InvalidDataException("Please enter a valid email.");
    }
    checkText(name);
    checkText(email);
    if (DataStore.emailExists(email.trim())) {
        throw new InvalidDataException("The email " + email + " is already used.");
    }
}
```

Teacher 3.4 `extends Person implements Authenticatable`

المدرس عنده `employeeId` (5 أرقام) و `password` . الـ methods بتاعته هي نفس خطوات الشغل اللي اتفقنا عليها:

Method	بتعمل إيه

`getCourses()`	بترجع الكورسات اللي الأدمن حطه مدرس ليها بس (بتسأل DataStore). عشان كده المدرس مش بيشوف كورسات غيره.
`startSession(course, type, details)`	**الخطوة 1.** تتأكد إن الكورس بتاعه، وبعدين على حسب النوع تعمل `LectureSession` أو `LabSession` أو `SectionSession`، وتضيفها للكورس. لاحظ إن نوع المتغير `AttendanceSession` بس الأوبجكت الحقيقي واحد من التلاتة: ده الـ **Polymorphism**.
`markAttendance(session, attendanceId)`	**الخطوة 2.** تدوّر على الطالب جوه طلبة الكورس. لو مش موجود خالص → "No student has the ID"." لو موجود بس مش متسجل في الكورس → "is not enrolled"." لو تمام تنادي `session.markPresent(s)` اللي بترفض لو اتسجل قبل كده.
`endSession(session)`	**الخطوة 3.** تقفل الجلسة (اللي ما اتسجلش يبقى غايب) وتحفظ في الملفات.

```
public AttendanceSession startSession(Course course, String type, String details)
        throws InvalidDataException {
    if (course.getTeacher() != this) {
        throw new InvalidDataException("You are not the teacher of this course.");
    }
    AttendanceSession session;
    if (type.equals("Lecture")) {
        session = new LectureSession(course, details);
    } else if (type.equals("Lab")) {
        session = new LabSession(course, details);
    } else {
        int group;
        try {
            group = Integer.parseInt(details.trim());
        } catch (NumberFormatException e) {
            throw new InvalidDataException("Group number must be a number.");
        }
        session = new SectionSession(course, group);
    }
    course.addSession(session);
    return session;
}
```

```
public Student markAttendance(AttendanceSession session, int attendanceId)
        throws InvalidDataException {
    Student s = session.getCourse().findStudent(attendanceId);
    if (s == null) {
        if (DataStore.findStudent(attendanceId) == null) {
            throw new InvalidDataException("No student has the ID " + attendanceId + ".");
        }
        throw new InvalidDataException("Student " + attendanceId + " is not enrolled in this
course.");
    }
    session.markPresent(s);
    return s;
}
```

Student 3.5 `<extends Person implements Comparable<Student`

فيه attribute واحد بس زيادة: `studentAttendanceId` (4 أرقام، من 1000 لـ 9999). ده الرقم اللي الطالب بيقوله للمدرس.

عامل `implements Comparable<Student>` (محاضرة 5) وعامل override لـ `compareTo` بحيث يقارن بالاسم. الفايدة: لما نعمل `Collections.sort(list)` الطلبة يترتبوا أبجديًا في شاشة الطلبة وفي التقارير.

```
public int compareTo(Student other) {
    return getName().compareToIgnoreCase(other.getName());
}
```

Course 3.6

الكورس هو "المكان" اللي بيجمع كل حاجة: كوده (unique زي COMP301)، اسمه، المدرس بتاعه (`Teacher`، ممكن يبقى null لحد ما الأدمن يحدده)، `ArrayList<Student>` للطلبة المسجلين، و `ArrayList<AttendanceSession>` لكل المحاضرات واللابات والسكاشن اللي اتعملت.

Method	بتعمل إيه
`enrollStudent(s)`	تضيف الطالب، وترمي exception لو كان متسجل قبل كده.
`removeStudent(s)` , `isEnrolled(s)`	شيل / سؤال هل متسجل.
`findStudent(attendanceId)`	تلف على طلبة الكورس بـ enhanced for loop وترجع اللي رقمه ده، أو null.
`addSession(session)` , `getSessions()`	إضافة / جلب الجلسات.
`toString()`	`COMP301 - Advanced Programming (Java)` .

قرار تصميم مهم: المدرس والطالب مش شايلين list بكورساتهم. الكورس بس هو اللي عارف مين مدرسه ومين طلبته. كده المعلومة متخزنة في مكان واحد، ومستحيل يحصل تناقض (زي إن الطالب يقول أنا في الكورس والكورس يقول لا).

```
public Student findStudent(int attendanceId) {
    for (Student s : students) {
        if (s.getStudentAttendanceId() == attendanceId) {
            return s;
        }
    }
    return null;
}
```

AttendanceSession 3.7 abstract class

دي "ورقة الحضور" بتاعة مرة واحدة: محاضرة واحدة أو لاب واحد أو سكشن واحد. وده هو تقسيم الحضور لتلاتة اللي طلبتوه.

العضو	معناه
static `- nextId: int`	عدّاد مشترك بين كل الجلسات (محاضرة 2 - static). كل جلسة جديدة تاخد الرقم اللي عليه وتزوّده 1.
`sessionId, course, date, closed`	رقم الجلسة، الكورس بتاعها، التاريخ (بيتاخد أوتوماتيك من `LocalDate.now()`)، وهل خلصت ولا لسه.
`records: ArrayList<AttendanceRecord>`	سطور الكشف: كل سطر طالب + حاضر/غايب.

abstract `getType()` , `getDetails()`	كل ابن بيرجع نوعه وتفاصيله.
`markPresent(s)`	ترفض لو الجلسة مقفولة أو الطالب متسجل قبل كده، وإلا تضيف record حاضر.
`isMarked(s)` , `isPresent(s)`	هل الطالب له سطر؟ وهل السطر ده حاضر؟
`close()`	أهم method: أي طالب مسجل في الكورس ومالوش سطر بيتضافله سطر **غايب**، وبعدين `closed = true` .
`countPresent()` , `countAbsent()`	عدد الحاضرين / الغايبين.
`addRecord(r)`	بتستخدم بس وقت تحميل الجلسات القديمة من الملف.

فيه كونستركتورين (Constructor Overloading): واحد للجلسة الجديدة (ياخد الكورس بس، والباقي يتحسب)، وواحد للجلسة اللي بتتقري من الملف (كل البيانات جاية جاهزة).

```
public void markPresent(Student s) throws InvalidDataException {
    if (closed) {
        throw new InvalidDataException("This session is already ended.");
    }
    if (isMarked(s)) {
        throw new InvalidDataException(s.getName() + " is already marked present.");
    }
    records.add(new AttendanceRecord(s, true));
}
```

```
public void close() {
    if (closed) {
        return;
    }
    for (Student s : course.getStudents()) {
        if (!isMarked(s)) {
            records.add(new AttendanceRecord(s, false));
        }
    }
    closed = true;
}
```

SectionSession g LabSession g LectureSession 3.8

التلاتة بيعملوا `extends AttendanceSession` . كل واحد بيضيف معلومة واحدة تخصه وبيعمل override لـ getType و getDetails:

الكلاس	المعلومة الزيادة	()getType	()getDetails
LectureSession	`topic: String` (موضوع المحاضرة)	"Lecture"	...Topic:
LabSession	`experiment: String` (عنوان التجربة/التاسك)	"Lab"	...Experiment:
SectionSession	`groupNumber: int` (رقم الجروب)	"Section"	...Group

ليه وراثة ومش مجرد متغير type؟ عشان كل نوع ليه بيانات مختلفة، وعشان التقارير تقدر تسأل كل جلسة `s.getType()` من غير ما تعرف هي أنهي نوع: الـ JVM هي اللي بتنادي النسخة الصح وقت التشغيل (Dynamic binding / Polymorphism).

```
package model;

/** A section (tutorial) session. Its extra detail is the group number. */
public class SectionSession extends AttendanceSession {

    private int groupNumber;

    public SectionSession(Course course, int groupNumber) {
        super(course);
        this.groupNumber = groupNumber;
    }

    public SectionSession(int id, Course course, String date, boolean closed, int groupNumber) {
        super(id, course, date, closed);
        this.groupNumber = groupNumber;
    }

    @Override
    public String getType() {
        return "Section";
    }

    @Override
    public String getDetails() {
        return "Group " + groupNumber;
    }

    public int getGroupNumber() {
        return groupNumber;
    }
}
```

AttendanceRecord 3.9

كلاس صغير جدًا: `student` و `present (boolean)` . بيمثل سطر واحد في الكشف. ليه مش بنخزن list بالحاضرين بس؟ لأن الطالب ممكن يتسجل في الكورس بعد جلسات قديمة، فلازم نعرف مين كان مسجل وقتها وغاب فعلًا.

AttendanceReport 3.10 static methods

مالوش attributes؛ كل الـ methods بتاعته static (بننادیها باسم الكلاس من غير new):

- `countSessions(c, type)` : عدد الجلسات المنتهية من نوع معين في الكورس.
- `countAttended(c, st, type)` : الطالب حضر كام واحدة منهم.
- `attendanceRate(c, st)` : نسبة الحضور الكلية = اللي حضرها × 100 ÷ كل الجلسات.
- `courseReport(c)` : جدول نصي لكل طلبة الكورس (Section x/y، Lab x/y، Lecture x/y، والنسبة). بيستخدم `StringBuffer` (محاضرة 4) و `String.format` عشان الأعمدة تبقى مظبوطة.
- `sessionReport(s)` : كشف جلسة واحدة: مين حضر ومين غاب.

```
public static double attendanceRate(Course c, Student st) {
    int total = 0;
    int attended = 0;
    String[] types = {"Lecture", "Lab", "Section"};
    for (String t : types) {
        total += countSessions(c, t);
        attended += countAttended(c, st, t);
    }
    if (total == 0) {
        return 0;
    }
    return attended * 100.0 / total;
}
```

3.11 InvalidDataException custom checked exception

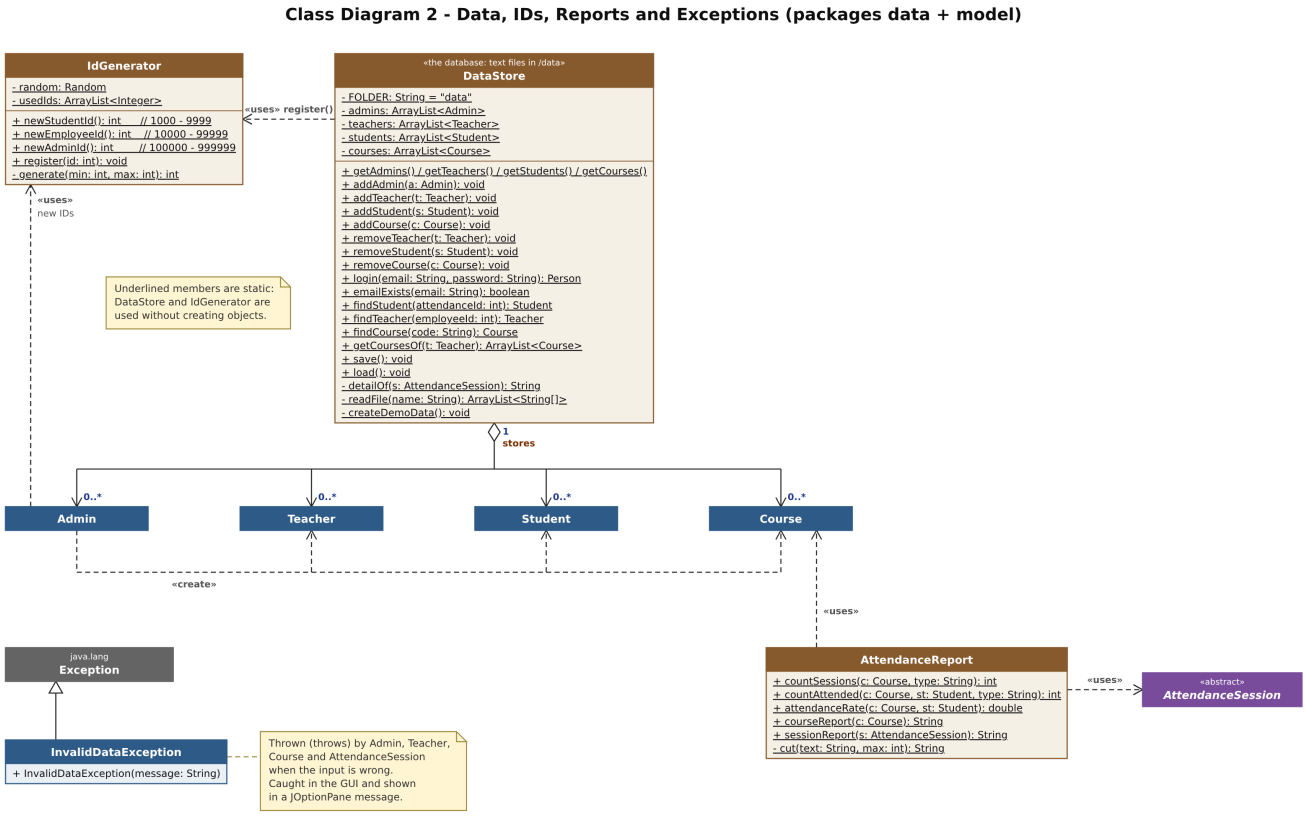
عامل `extends Exception` (مش RuntimeException)، يعني checked: أي method بترميه لازم تكتب `throws`، واللي بيناديها لازم يعمل try/catch (محاضرة 6). بنرميه في كل حالة إدخال غلط، والشاشة بتمسكه وتعرض رسالته في `JOptionPane`. كده البرنامج عمره ما يقع بسبب إدخال غلط.

```
package model;

/**
 * Custom checked exception (Lecture 6 - Custom Exceptions).
 * Thrown whenever the user enters something wrong, for example:
 * an attendance ID that does not exist, a student marked twice,
 * an email that is already used, an empty name ...
 * The GUI catches it and shows its message to the user.
 */
public class InvalidDataException extends Exception {

    public InvalidDataException(String message) {
        super(message);
    }
}
```

4. الـ package اللي اسمها data (الداتا بيز)



Class Diagram 2

4.1 ليه ملفات نصية ومش Database حقيقية؟

المنهج (المحاضرات 1 لـ 9) مفيهوش أي حاجة عن الـ Database أو JDBC أو SQL. اللي فيه هو **File Handling** في محاضرة 6: `FileReader` و `BufferedReader` و `FileWriter` و `BufferedWriter`. فعملنا الداتا بيز عبارة عن 5 ملفات نصية، كل ملف كأنه جدول وكل سطر كأنه صف. كده مش محتاجين أي مكتبة ولا ربط يدوي، والمشروع يشتغل على أي جهاز عليه Java.

الملف (الجدول)	شكل السطر	مثال
admins.txt	`id;name;email;password`	`608034;System Admin;admin@ams.com;admin123`
teachers.txt	`id;name;email;password`	`29573;Dr. Hany Samir;hany@ams.com;1234`
students.txt	`id;name;email`	`6280;Mohamed Ali;student1@ams.com`
courses.txt	`code;name;teacherId;studentIds`	`COMP301;Advanced Programming (Java);29573;6280,7688,7556`

sessions.txt	`id;course;type;date;detail;records`	`1;COMP301;Lecture;2026-10-12;Inheritance;6280:1,8616:0` (1 = حاضر، 0 = غايب)

لاحظوا إن الكورس بيخزن **أرقام** المدرس والطلبة مش بياناتهم كلها (زي الـ foreign key في الداتا بيز). ولما نقرا الملف بنرجع نجيب الأوبجكت بالرقم.

DataStore 4.2

كل حاجة فيه `static` ، فأي كلاس يقدر يستخدمه من غير ما يعمل object. جواه 4 قوايم: `admins` و `teachers` و `students` و `courses` (ArrayList من محاضرة 3). الجلسات مش محتاجة قايمة لوحدها لأنها جوه الكورسات.

Method	بتعمل إيه
`addX(...)`	تضيف في القايمة وتنادي `save()` .
`removeStudent(s)`	تشيله من القايمة ومن كل الكورسات.
`removeTeacher(t)`	تشيله وتخلي كورساته من غير مدرس (null) بدل ما تمسحها.
`login(email, password)`	تلف على الأدمنز وبعدين المدرسين وتنادي `login` بتاعة كل واحد. ترجع الشخص أو null.
`emailExists(email)`	تجمع الكل في list واحدة من نوع `Person` (Polymorphism) وتشوف الإيميل متكرر ولا لا.
`findStudent / findTeacher / findCourse`	بحث بالرقم أو الكود.
`getCoursesOf(teacher)`	الكورسات اللي مدرسها ده.
`save()`	تكتب الـ 5 ملفات من الأول بـ BufferedWriter. بتحفظ الجلسات **المنتهية** بس.
`load()`	لو أول مرة (مفيش admins.txt) تعمل داتا تجريبية. غير كده تقرا الملفات بالترتيب: admins، teachers، students، courses، sessions، وتسجّل كل ID في IdGenerator.
`readFile(name)` (private)	تقرا ملف سطر سطر بـ `readLine()` وتقسم كل سطر بـ `split(";", -1)` . الـ -1 عشان لو آخر خانة فاضية (كورس مفيهوش طلبة) متضيعش.
`createDemoData()` (private)	أدمن + 2 مدرسين + كورسين + 5 طلبة، عشان تقدروا تجربوا على طول.

```
private static ArrayList<String[]> readFile(String name) throws IOException {
    ArrayList<String[]> lines = new ArrayList<String[]>();
    File f = new File(FOLDER + "/" + name);
    if (!f.exists()) {
        return lines;
    }
    BufferedReader r = new BufferedReader(new FileReader(f));
    String line;
    while ((line = r.readLine()) != null) {
        if (!line.trim().isEmpty()) {
            lines.add(line.split(";", -1));   // -1 keeps empty last parts
        }
    }
    r.close();
    return lines;
}
```

```
w = new BufferedWriter(new FileWriter(FOLDER + "/students.txt"));
for (Student s : students) {
    w.write(s.getId() + ";" + s.getName() + ";" + s.getEmail());
    w.newLine();
}
w.close();
```

IdGenerator 4.3

مسؤول عن الأرقام العشوائية اللي مش بتتكرر. جواه `Random` و `ArrayList<Integer> usedIds` فيها كل رقم اتاخد قبل كده.

- `newStudentId()` ← بين 1000 و 9999 (4 أرقام).
- `newEmployeeId()` ← بين 10000 و 99999 (5 أرقام).
- `newAdminId()` ← بين 100000 و 999999 (6 أرقام).
- DataStore ← `register(id)` بتناديها وقت التحميل عشان الأرقام القديمة متتكررش.

الفكرة كلها في `generate`: اختار رقم عشوائي في المدى، ولو موجود قبل كده اختار تاني (do-while)، وبعدين سجّله:

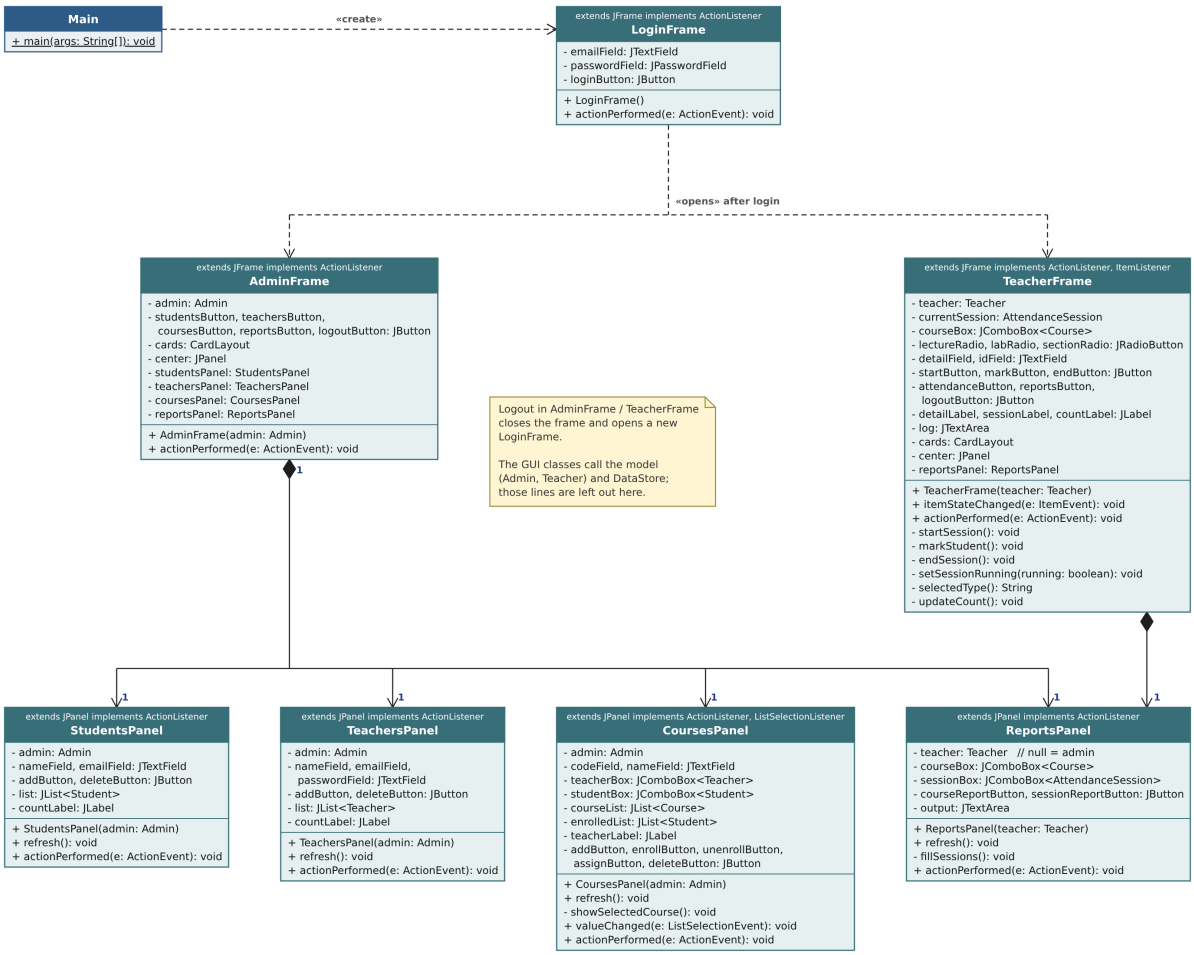
```
private static int generate(int min, int max) {
    int id;
    do {
        id = min + random.nextInt(max - min + 1);
    } while (usedIds.contains(id));
    usedIds.add(id);
    return id;
}
```

`random.nextInt(n)` بيرجع رقم من 0 لـ n-1، فـ `min + nextInt(max-min+1)` بيدينا رقم من min لـ max بالظبط.

5. الـ package اللي اسمها gui (الشاشات)

كل الشاشات Swing (محاضرة 7) والأحداث بنفس طريقة محاضرة 8: الكلاس نفسه بيعمل `implements ActionListener`، وكل زرار بنعمله `addActionListener(this)`، وجوه `actionPerformed` بنسأل `e.getSource()` عشان نعرف مين اتداس. مستخدمناش lambdas لأنها مش في المنهج.

Class Diagram 3 - GUI (package gui, Java Swing)

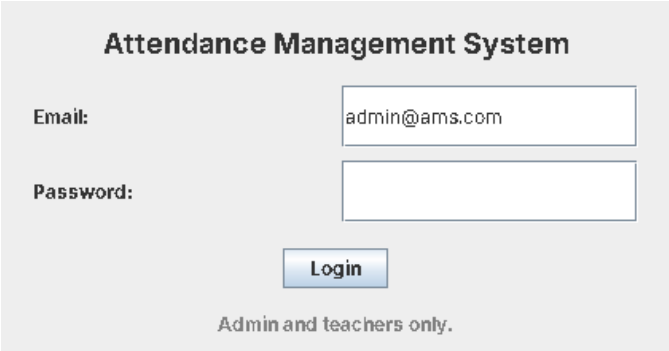


Class Diagram 3: الشاشات

Main 5.1

سطرين: `DataStore.load()` عشان نحمّل البيانات، وبعدين `new LoginFrame()`. بالظبط زي مثال `new SwingDemo()` في المحاضرة.

LoginFrame 5.2



شاشة الدخول

`extends JFrame implements ActionListener`. فيها JTextField للإيميل و JPasswordField للباسورد (بيخفي الحروف) وزرار Login. الـ layout: BorderLayout للفريم، و GridLayout(2,2) للفورم.

لما تدوس Login (أو Enter في خانة الباسورد): تنادي `DataStore.login`. لو null تطلع رسالة خطأ. لو الأوبجكت `instanceof Admin` تفتح AdminFrame، غير كده TeacherFrame، وبعدين `dispose()` تقفل شاشة الدخول.

```
public void actionPerformed(ActionEvent e) {
    String email = emailField.getText().trim();
    String password = new String(passwordField.getPassword());

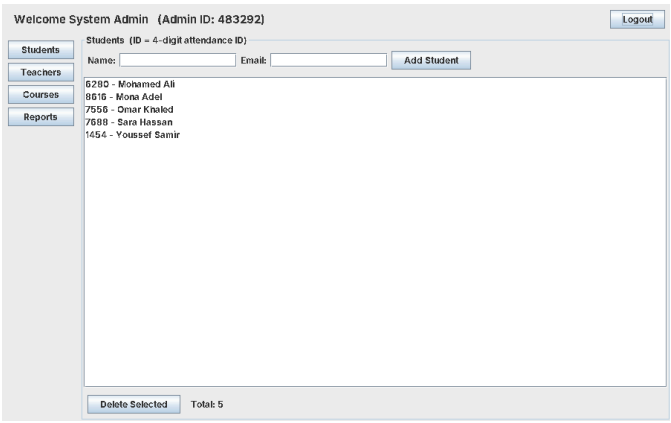
    Person user = DataStore.login(email, password);
    if (user == null) {
        JOptionPane.showMessageDialog(this, "Wrong email or password.", "Login failed",
                JOptionPane.ERROR_MESSAGE);
        return;
    }
    if (user instanceof Admin) {
        new AdminFrame((Admin) user);
    } else {
        new TeacherFrame((Teacher) user);
    }
    dispose();   // close the login window
}
```

AdminFrame 5.3

فوق: ترحيب ورقم الأدمن وزرار Logout. على الشمال: 4 زرايز (Students, Teachers, Courses, Reports). في النص: JPanel عامل `CardLayout` (من الـ layouts المذكورة في محاضرة 7) فيها 4 كروت؛ لما تدوس زرار بنعمل `refresh()` للكارت وبعدين `cards.show(...)`.

```
public void actionPerformed(ActionEvent e) {
    Object src = e.getSource();
    if (src == studentsButton) {
        studentsPanel.refresh();
        cards.show(center, "students");
    } else if (src == teachersButton) {
        teachersPanel.refresh();
        cards.show(center, "teachers");
    } else if (src == coursesButton) {
        coursesPanel.refresh();
        cards.show(center, "courses");
    } else if (src == reportsButton) {
        reportsPanel.refresh();
        cards.show(center, "reports");
    } else if (src == logoutButton) {
        dispose();
        new LoginFrame();
    }
}
```

TeachersPanel g StudentsPanel 5.4



كارت الطلبة

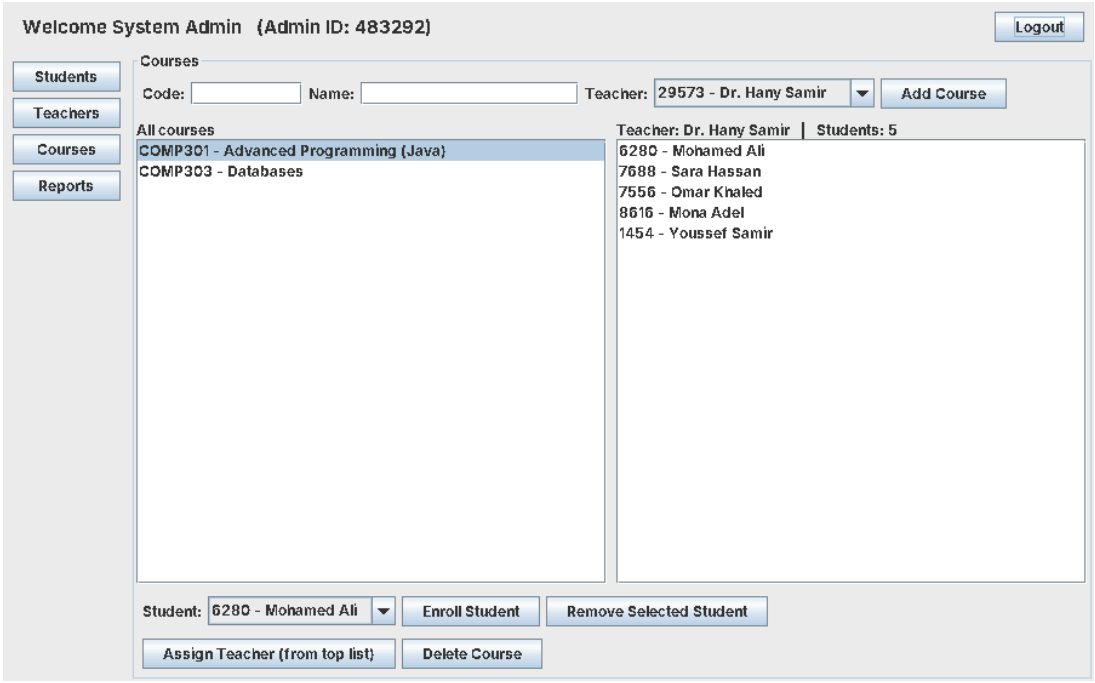
فوق فورم (FlowLayout) فيه الاسم والإيميل (وللمدرس الباسورد) وزرار Add. في النص `JList` جوه `JScrollPane`. تحت زرار Delete وعدد الكل.

Add: بتنادي `admin.createStudent(...)` جوه try/catch. لو تمام تطلع رسالة فيها **رقم الحضور الجديد**. لو غلط تطلع رسالة الـ exception.

Delete: تاخد `list.getSelectedValue()` وتسأل تأكيد بـ `JOptionPane.showConfirmDialog`.

`refresh()` : بتعمل نسخة من قايمة الطلبة، ترتبها بـ `Collections.sort` (بسبب compareTo)، وتحطها في الـ JList بـ `setListData`.

CoursesPanel 5.5

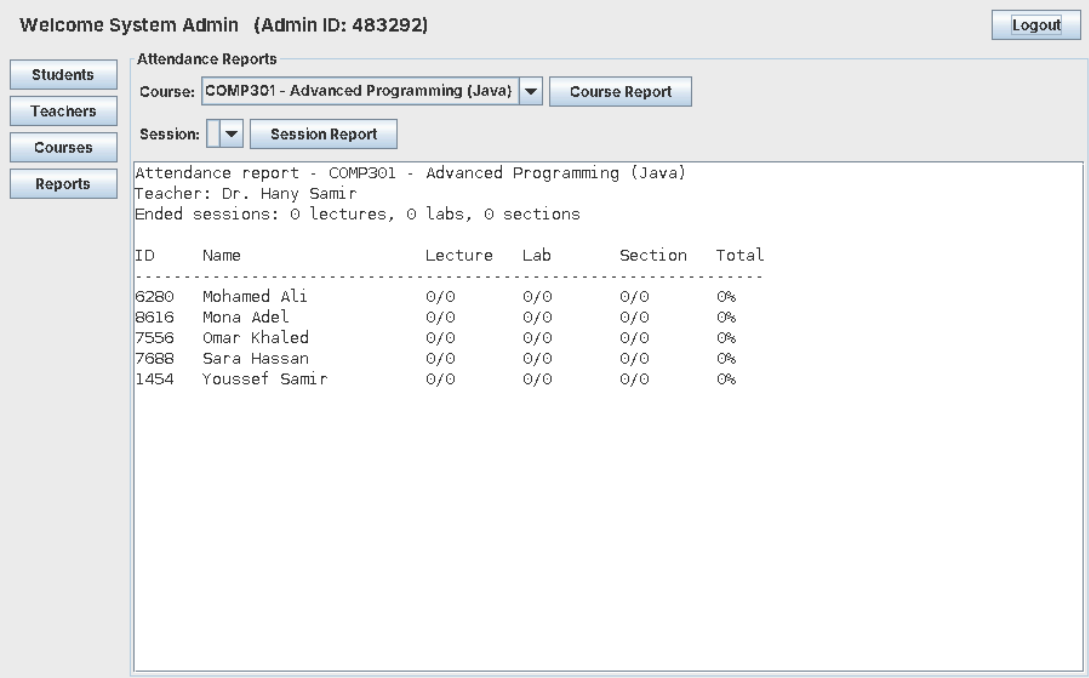


كارت الكورسات

فوق: فورم الكورس الجديد (Code, Name, Teacher في JComboBox). في النص قايمتين جنب بعض ((GridLayout(1,2): على الشمال كل الكورسات، وعلى اليمين طلبة الكورس المختار. القايمة الشمال عاملة `addListSelectionListener(this)`، فلما تختار كورس method اسمها `valueChanged` بتشتغل وتعرض طلبته ومدرسه (ListSelectionListener من محاضرة 8).

تحت: اختار طالب من JComboBox ودوس **Enroll Student**، أو اختار طالب من القايمة اليمين ودوس **Remove Selected Student**، أو **Assign Teacher** (بياخد المدرس من الـ combo اللي فوق)، أو **Delete Course**.

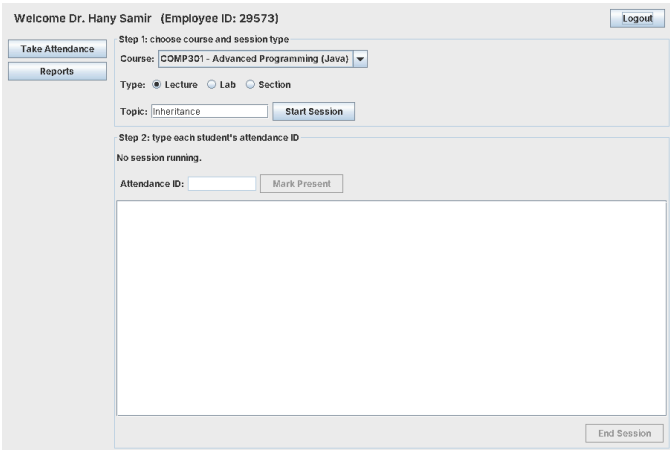
ReportsPanel 5.6



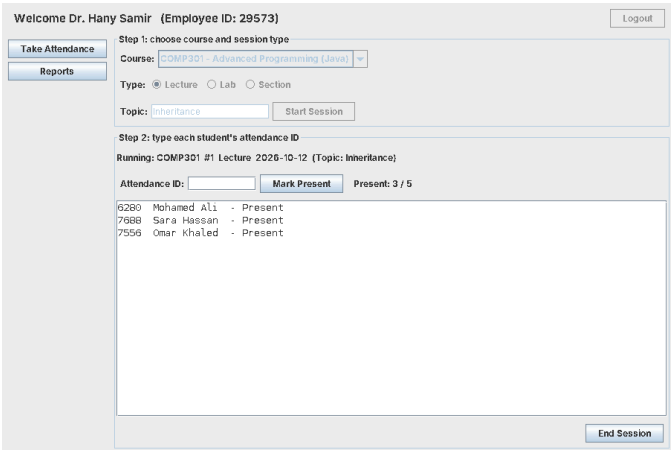
التقارير عند الأدمن

نفس الكلاس بيستخدمه الأدمن والمدرس. الكونستركتور بياخد `Teacher` : لو null يبقى أدمن ويشوف كل الكورسات، غير كده يشوف كورسات المدرس ده بس. لما تختار كورس من الـ combo (هو كمان بيعمل ActionEvent) بتتملى قايمة الجلسات المنتهية. **Course Report** و **Session Report** بيكتبوا النتيجة في JTextArea بخط `Monospaced` (Font من محاضرة 9) عشان الأعمدة تبقى تحت بعض.

TeacherFrame 5.7 (أهم شاشة)



الخطوة 1



الخطوة 2 أثناء التسجيل

`extends JFrame implements ActionListener, ItemListener` . فيها كارتين بـ CardLayout: Take Attendance و Reports.

- JComboBox **Step 1:** بكورسات المدرس، و3 JRadioButton (Lecture/Lab/Section) جوه `ButtonGroup` عشان تختار واحد بس (محاضرة 7)، وخانة للتفاصيل. الراديو عامل `addItemListener(this)` ، فلما تغير النوع method `itemStateChanged` بتغير اسم الخانة لـ Topic أو Experiment أو Group number.

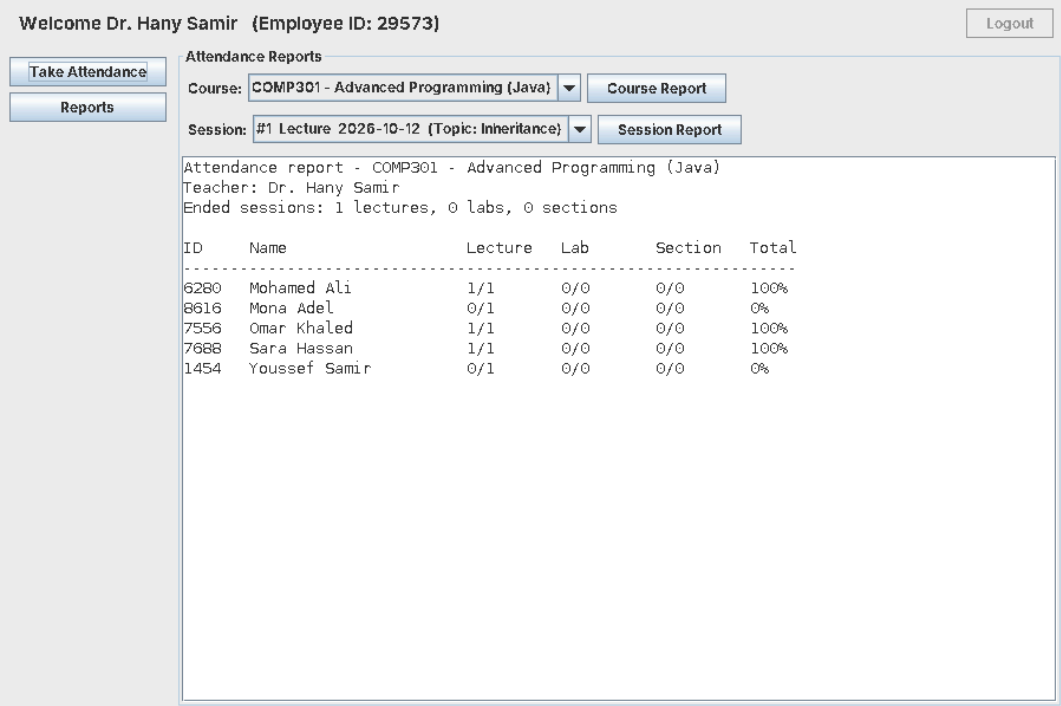
- **Start Session:** تتأكد إن التفاصيل مكتوبة، وتنادي `teacher.startSession(...)`، وبعدين `setSessionRunning(true)` اللي بتقفل الخطوة 1 وتفتح الخطوة 2.
- **Step 2:** المدرس يكتب الرقم ويدوس Mark Present أو Enter (عشان الـ JTextField نفسه عامل addActionListener). الرقم بيتحول بـ `Integer.parseInt`، ولو مش رقم بنمسك `NumberFormatException`. كل طالب يتسجل بيظهر سطر في الـ JTextArea، والعداد "Present: x / y" يتحدّث.
- **End Session:** تأكيد، وبعدين `teacher.endSession(...)`، ويعرض كشف الجلسة كامل (الحاضرين والغايبين).
- وأثناء الجلسة زرار Logout مقفول عشان محدش يخرج والجلسة مفتوحة.

```
private void markStudent() throws InvalidDataException {
    int id;
    try {
        id = Integer.parseInt(idField.getText().trim());
    } catch (NumberFormatException ex) {
        throw new InvalidDataException("The attendance ID must be a 4-digit number.");
    }
    Student s = teacher.markAttendance(currentSession, id);
    log.append(s.getId() + "  " + s.getName() + "  - Present\n");
    idField.setText("");
    idField.requestFocus();
    updateCount();
}
```

```
public void itemStateChanged(ItemEvent e) {
    if (lectureRadio.isSelected()) {
        detailLabel.setText("Topic:");
    } else if (labRadio.isSelected()) {
        detailLabel.setText("Experiment:");
    } else {
        detailLabel.setText("Group number:");
    }
}
```

6. سيناريو كامل: إيه اللي بيحصل جوه البرنامج

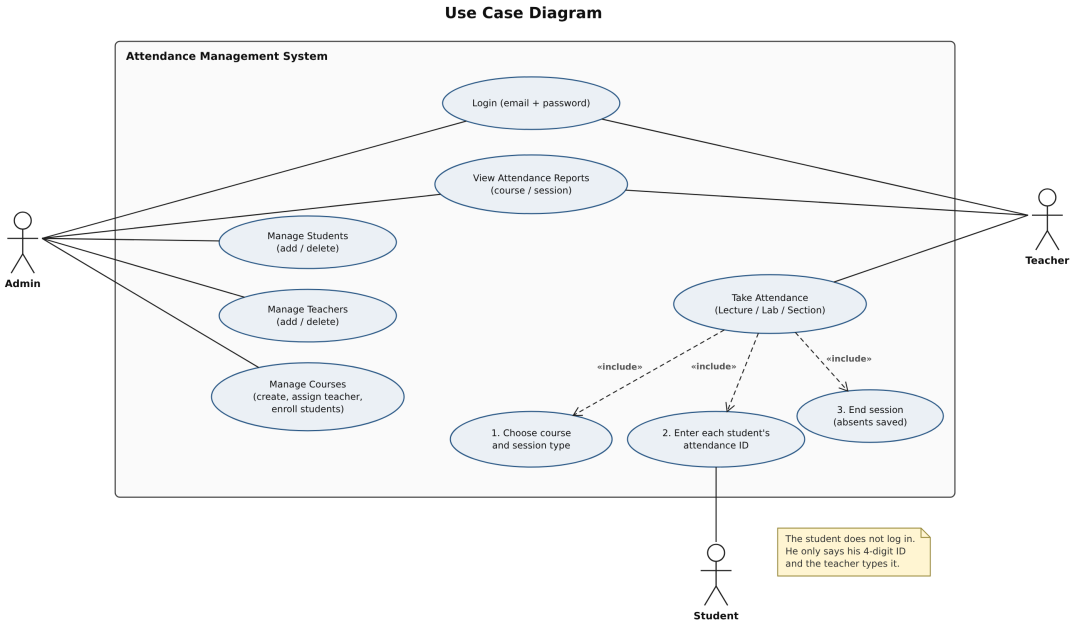
#	المستخدم بيعمل	اللي بيحصل في الكود
1	يشغّل البرنامج	`Main.main` → `DataStore.load()` (أول مرة: `createDemoData()`) → `new LoginFrame()`
2	الأدمن يدخل admin@ams.com / admin123	`Admin.login` → `DataStore.login` ترجع `new AdminFrame(admin)` → true
3	يضيف طالب "Ali Hassan"	`StudentsPanel.actionPerformed` → `admin.createStudent` → `IdGenerator.newStudentId()` مثلًا 4821 → `DataStore.addStudent` → `save()`
4	يسجله في COMP301	`CoursesPanel` → `admin.enrollStudent` → `course.enrollStudent` → `save()`
5	المدرس يدخل بإيميله	`Teacher.login` → `new TeacherFrame(teacher)` → الـ combo بيتملى من `teacher.getCourses()`
6	يختار COMP301 و Lab ويكتب "Swing" ويدوس Start	`teacher.startSession(c, "Lab", "Swing")` → `new LabSession(c, "Swing")` بـ sessionId من `nextId` → `course.addSession`
7	Ali يقول 4821 والمدرس يكتبه	`teacher.markAttendance(session, 4821)` → `course.findStudent` حاضر → `session.markPresent` → record
8	حد يقول رقم غلط 1234	`InvalidDataException("No student has the ID 1234.")` → رسالة خطأ، والجلسة شغالة عادي
9	End Session	`teacher.endSession` → `session.close()` (الباقي غياب) → `DataStore.save()` → sessions.txt
10	Reports → Course Report	`AttendanceReport.courseReport(course)` → جدول بالنسب



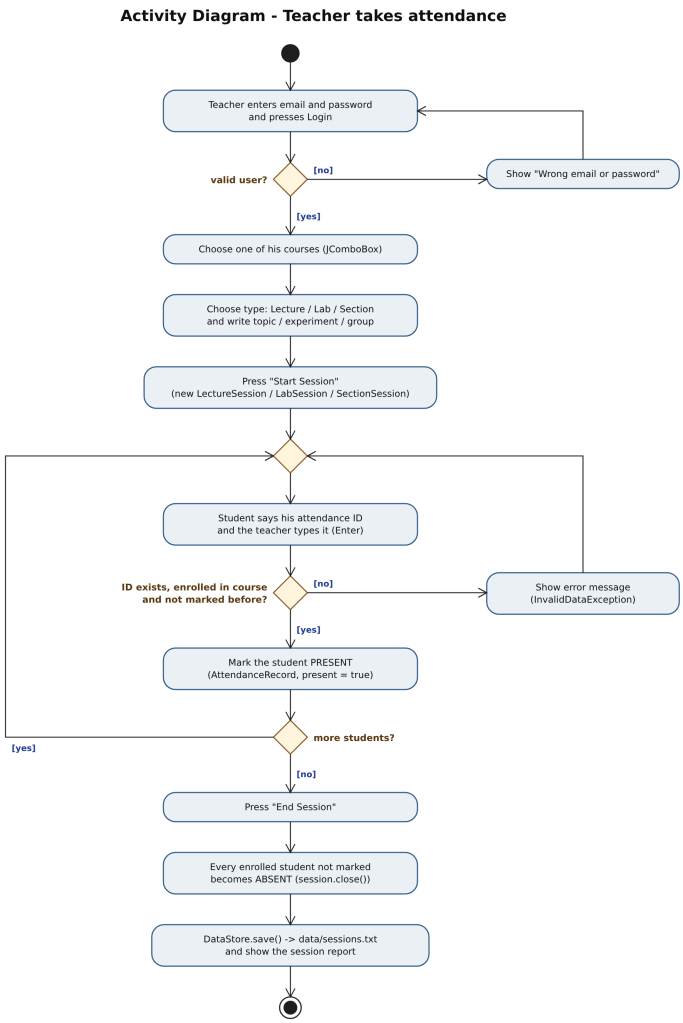
تقرير الكورس عند المدرس

7. إزاي تقروا خطوط الـ UML

الخط	معناه وأمثلة من المشروع
خط متصل وفي آخره مثلث فاضي	**Inheritance (extends)**: Admin/Teacher/Student → Person، Lecture/Lab/SectionSessiong → AttendanceSession. السهم بيشاور على الأب.
خط متقطع ومثلث فاضي	**Realization (implements)**: Admin و Teacher → Authenticatable، و Student → Comparable.
معين مليان ◆	**Composition**: الجزء ميعيشش من غير الكل. الجلسة ملهاش لازمة من غير الكورس، والـ record ملوش لازمة من غير الجلسة. لو مسحت الكورس تتمسح جلساته.
معين فاضي ◇	**Aggregation**: الكورس فيه طلبة، بس لو مسحت الكورس الطلبة موجودين. وكمان DataStore شايل كل الأوبجكتس.
خط متصل وسهم مفتوح	**Association**، والسهم ناحية الكلاس اللي الطرف التاني شايل reference ليه: Course عنده attribute اسمه teacher، فالسهم ناحية Teacher.
خط متقطع وسهم مفتوح	**Dependency**: كلاس بيستخدم أو بيعمل كلاس تاني من غير ما يشيله كـ attribute، زي Admin «create» Student، «uses» IdGeneratorg Admin.
الأرقام 1, 0..1, 0..*	**Multiplicity**: الكورس له 0..1 مدرس (ممكن ميكونش اتحدد لسه)، والمدرس له 0..* كورسات. الطالب في 0..* كورسات والكورس فيه 0..* طلبة.
- + في أول السطر	public `+`، private `-`. الكلام *المايل* abstract، واللي تحته خط static.



Use Case Diagram



Activity Diagram لفلو المدرس

8. أسئلة متوقعة في المناقشة

السؤال	الإجابة
ليه Person abstract؟	لأن مفيش شخص في السيستم مش أدمن ولا مدرس ولا طالب، فمش منطقي نعمل منه object. وفيه methods (getId, getRole) كل ابن لازم يكتبها بطريقته.
إيه الفرق بين abstract class و interface في مشروعكم؟	Person فيها بيانات (name, email) وكود مشترك، فبقت abstract class. Authenticatable مجرد "قدرة" (login) بيعملها بعض الكلاسات بس، فبقت interface. والكلاس بيورث من أب واحد بس لكن يقدر يعمل implements لكذا interface.
فين الـ Polymorphism؟	(1) `AttendanceSession session = new LabSession(...)` و `session.getType()` بترجع "Lab". (2) `ArrayList<Person>` في emailExists فيها أدمنز ومدرسين وطلبة. (3) `toString()` بتاعة كل كلاس.
إزاي ضمنتوا إن الـ ID مش بيتكرر؟	IdGenerator شايل list بكل رقم اتاخد، ولو الرقم العشوائي موجود يختار غيره (do-while). وكمان كل الأرقام القديمة بتتسجل فيه وقت تحميل الملفات.
ليه الأرقام بأعداد خانات مختلفة؟	عشان من عدد الأرقام تعرف هو مين (4 طالب، 5 مدرس، 6 أدمن)، والمدى مختلف فمستحيل يتداخلوا.
ليه الطالب مش بيعمل login؟	ده المطلوب: المدرس هو اللي بيسجل. وفي التصميم Student مش بيعمل implements لـ Authenticatable.
لو المدرس كتب رقم طالب مش في الكورس؟	markAttendance بترمي InvalidDataException برسالة واضحة، والشاشة بتعرضها وتكمل عادي.
لو كتب نفس الرقم مرتين؟	markPresent بتلاقي إن الطالب isMarked فترمي exception "already marked present".
الغياب بيتسجل إمتى؟	وقت close: End Session()) بتلف على طلبة الكورس واللي مالوش سطر بيتضافله سطر present = false.
البيانات بتتحفظ فين؟ وليه مش database؟	في 5 ملفات txt جوه فولدر data. المنهج فيه File Handling (محاضرة 6) ومفيهوش JDBC، فاستخدمنا BufferedReader/BufferedWriter.
Checked ولا Unchecked exception؟ وليه؟	Checked (extends Exception) عشان الكومبايلر يجبر أي حد بينادي الـ method إنه يتعامل مع الخطأ، فمستحيل ننسى نعرض الرسالة.
فين الـ static وليه؟	DataStore و IdGenerator و AttendanceReport: مفيش داعي يبقى منهم أكتر من نسخة. و nextId في AttendanceSession عشان يبقى عداد مشترك.
استخدمتوا أنهي Layouts و Listeners؟	BorderLayout و FlowLayout و GridLayout و CardLayout. و ActionListener و ItemListener (الراديو) و ListSelectionListener (قايمة الكورسات).
ليه Course هو اللي شايل المدرس والطلبة؟	عشان المعلومة تبقى في مكان واحد ومفيش تناقض. ولو عايزين كورسات المدرس بنسأل DataStore.getCoursesOf.
إيه فايدة Comparable؟	ترتيب الطلبة بالاسم بـ Collections.sort في شاشة الطلبة وفي التقرير.

9. الحسابات الجاهزة للتجربة

الدور	الإيميل	الباسورد
Admin	admin@ams.com	admin123
Teacher (COMP301)	hany@ams.com	1234
Teacher (COMP303)	ahmed@ams.com	1234

أرقام الطلبة بتطلع عشوائي أول تشغيل؛ هتلاقوها في شاشة Students عند الأدمن أو في ملف `data/students.txt` .
ولو عايزين تبدأوا من الأول امسحوا فولدر `data` وشغّلوا تاني.